

Taller Angular 2 — Preguntas Conceptuales

ISIS2603 Desarrollo de Software en Equipos — Universidad de los Andes

1. Sobre Observables y Asincronía

¿Por qué las peticiones HTTP en Angular devuelven un Observable en lugar de la data directa? ¿Qué diferencia hay frente a una Promise?

Las peticiones HTTP en Angular devuelven un Observable porque este modelo es más flexible y poderoso para manejar operaciones asíncronas. A diferencia de una Promise, que se ejecuta una sola vez y no puede cancelarse, un Observable permite cancelar la suscripción en cualquier momento, transformar los datos usando operadores como map o catchError, y en teoría emitir múltiples valores a lo largo del tiempo. Esto hace que los Observables sean ideales para escenarios donde se necesita control total sobre el flujo de datos, como en aplicaciones reactivas.

¿Qué ocurre si un componente llama a un servicio HTTP pero nunca hace .subscribe()? ¿Se ejecuta la petición igualmente?

No, la petición no se ejecuta. Los Observables en Angular son 'lazy' (perezosos), lo que significa que el código dentro del Observable solo se activa cuando alguien se suscribe con .subscribe(). Si un componente obtiene un Observable de un servicio pero nunca lo suscribe, la petición HTTP nunca se enviará al servidor. Es similar a tener el cable enchufado pero sin encender el interruptor.

2. Sobre Inyección de Dependencias

¿Qué ventaja de modularidad da providedIn: 'root' frente a instanciar el servicio manualmente con new CityService() dentro de un componente?

Con providedIn: 'root', Angular crea una única instancia del servicio para toda la aplicación (patrón Singleton) y la inyecta automáticamente donde se necesite. Esto permite compartir estado entre componentes, reduce el uso de memoria y facilita el mantenimiento. En cambio, usar new CityService() dentro de un componente crea una instancia independiente por cada componente, lo que impide compartir datos, duplica recursos y acopla fuertemente el componente con la implementación concreta del servicio.

¿Cómo facilitaría la inyección de dependencias la escritura de pruebas unitarias para estos servicios?

La inyección de dependencias permite sustituir fácilmente los servicios reales por versiones simuladas (mocks) durante las pruebas. Por ejemplo, en lugar de usar el CityService real que hace peticiones HTTP al backend, se puede inyectar un mock que devuelve datos predefinidos. Esto hace que las pruebas sean rápidas, predecibles y no dependan de un servidor real, logrando un verdadero aislamiento de la unidad bajo prueba.

3. Sobre Interceptores

¿Por qué es mejor centralizar el manejo de errores en `HttpInterceptor` en lugar de poner bloques `try/catch` en cada componente?

Centralizar el manejo de errores en un `HttpInterceptor` aplica el principio DRY (Don't Repeat Yourself). Si se maneja el error en cada componente, se repite el mismo código en decenas de lugares, lo que aumenta la posibilidad de inconsistencias y dificulta el mantenimiento. Con el interceptor, cualquier cambio en la lógica de manejo de errores se hace en un solo lugar y se aplica automáticamente a todas las peticiones HTTP de la aplicación.

Además del manejo de errores, mencione dos casos de uso adicionales para un interceptor.

1. Autenticación automática: El interceptor puede agregar automáticamente el header `Authorization: Bearer <token>` a todas las peticiones salientes, evitando tener que incluirlo manualmente en cada servicio. 2. Indicador global de carga: El interceptor puede activar un spinner o barra de progreso cuando comienza cualquier petición HTTP y desactivarlo cuando termina, proporcionando retroalimentación visual al usuario sin necesidad de lógica adicional en cada componente.

4. Sobre el Patrón Maestro-Detalle

¿Por qué se usa `@Input()` para pasar la ciudad a `CityDetailComponent` en lugar de volver a hacer un GET a `/api/cities/{id}`?

Porque el componente padre (`CityListComponent`) ya tiene la información de la ciudad en memoria. Hacer un GET adicional implicaría una petición de red innecesaria al servidor, aumentando la latencia y la carga del backend. Usando `@Input()`, los datos fluyen directamente del componente padre al hijo sin costo adicional, siguiendo el principio de eficiencia y el patrón de comunicación entre componentes de Angular.

Observe que en la respuesta de GET `/api/cities`, el campo `country` ya viene como objeto anidado (no solo el `countryId`). ¿Qué ventaja de diseño tiene esta decisión del backend para el frontend?

Esta decisión implementa el patrón de respuesta enriquecida, donde el backend incluye los datos relacionados en una sola respuesta. La ventaja para el frontend es que con una única petición GET `/api/cities` obtiene toda la información necesaria para mostrar la tabla completa (ID, nombre de la ciudad y nombre del país), sin necesidad de hacer peticiones adicionales por cada ciudad para obtener el nombre del país. Esto reduce significativamente el número de llamadas al servidor y mejora el rendimiento de la aplicación.